```
warn: Microsoft.AspNetCore.HttpsPolicy.HttpsRedirectionMiddleware[3]
      Failed to determine the https port for redirect.
```

If you want the middleware to redirect to a different port than Kestrel knows about, you can configure that by setting the `ASPNETCORE_HTTPS_PORT` environment variable. This is sometimes necessary if you're using a reverse proxy, and it can be set in alternative ways, as described in Microsoft's "Enforce HTTPS in ASP.NET Core" documentation: http://mng.bz/6DDA.

SSL/TLS offloading, header forwarding, and detecting secure requests

At the start of section 28.1 I encouraged you to consider terminating HTTPS requests at a reverse proxy. That way, the user uses HTTPS to talk to the reverse proxy, and the reverse proxy talks to your app using HTTP. With this setup, your users are protected, but your app doesn't have to deal with TLS certificates itself.

For the `HttpsRedirectionMiddleware` to work correctly, Kestrel needs some way of knowing whether the original request that the reverse proxy received was over HTTP or HTTPS. The reverse proxy communicates to your app over HTTP, so Kestrel can't figure that out without extra help.

The standard approach used by most reverse proxies (such as IIS, NGINX, and HAProxy) is to add headers to the request before forwarding it to your app. Specifically, a header called `X-Forwarded-Proto` is added, indicating whether the original request protocol was HTTP or HTTPS.

ASP.NET Core includes `ForwardedHeadersMiddleware` to look for this header (and others) and update the request accordingly, so your app treats a request that was originally secured by HTTPS as secure for all intents and purposes.

If you're using IIS with the `UseIisIntegration()` extension, the header forwarding is handled for you automatically. If you're using a different reverse proxy, such as NGINX or HAProxy, you can enable the middleware by setting the environment variable